

Laboratorio 4: *Contenedores y Despliegue con Docker*

Santiago Palacio Cárdenas

Juan Esteban Mosquera Perea

Maria Paula Mosquera Álvarez

Docente: Juan Pablo Arango

Asignatura: Cloud Computing

Facultad de Ingeniería

Medellín, 2026

Contenido

1. Introducción 3
2. Objetivos 3
2.1 Objetivo general 3
2.2 Objetivos específicos 3
3. Marco teórico 3
4. Herramientas y entorno 4
5. Desarrollo del laboratorio 4
5.1 Creación de la aplicación 4
5.2 Creación del Dockerfile 6
5.3 Construcción de la imagen 7
5.4 Ejecución del contenedor 8
5.5 Prueba del servicio 8
5.6 Publicación en Docker Hub 9
6. Cómo Docker mejora la arquitectura de despliegue 10
7. Conclusiones 11
8. Referencias 12

1. Introducción

En el desarrollo de software moderno es común que una aplicación funcione en el equipo del desarrollador pero falle en otro entorno por diferencias en versiones, librerías o configuración. La contenerización resuelve este problema empaquetando la aplicación junto con todo lo que necesita para ejecutarse.

En este laboratorio se desarrolló una aplicación web sencilla en Node.js, se empaquetó en una imagen Docker mediante un Dockerfile, se ejecutó como contenedor exponiendo un servicio HTTP y, finalmente, se publicó la imagen en Docker Hub para que pueda descargarse y ejecutarse en cualquier equipo.

2. Objetivos

2.1 Objetivo general

Comprender el concepto de contenerización y aplicar Docker para empaquetar, ejecutar y distribuir una aplicación en un entorno aislado y reproducible.

2.2 Objetivos específicos

1. Instalar y configurar Docker correctamente.
2. Comprender la diferencia entre imágenes y contenedores.
3. Crear un Dockerfile para construir una imagen personalizada.
4. Ejecutar un contenedor localmente y exponer un servicio mediante un puerto.
5. Utilizar comandos básicos de Docker para gestionar imágenes y contenedores.
6. Subir la imagen a Docker Hub.

3. Marco teórico

Docker: plataforma que permite empaquetar aplicaciones y sus dependencias en unidades aisladas llamadas contenedores. A diferencia de las máquinas virtuales, los contenedores comparten el kernel del sistema operativo anfitrión, por lo que son más livianos y arrancan en segundos.

Imagen: plantilla inmutable y de solo lectura que contiene el sistema base, el entorno de ejecución, las dependencias y el código de la aplicación. Se construye por capas a partir de un Dockerfile.

Contenedor: instancia en ejecución de una imagen, con sus propios procesos, red y sistema de archivos. A partir de una misma imagen se pueden crear muchos contenedores.

Dockerfile: archivo de texto con las instrucciones (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD, etc.) que Docker sigue para construir una imagen.

Docker Hub: registro público en la nube donde se almacenan y comparten imágenes. Funciona de forma similar a GitHub, pero para imágenes Docker, mediante los comandos docker push y docker pull.

4. Herramientas y entorno

Herramienta	Uso en el laboratorio
Docker Desktop 29.4.1	Motor de Docker para construir imágenes y ejecutar contenedores en macOS.
Node.js 20 (node:20-alpine)	Entorno de ejecución de la aplicación dentro del contenedor.
Express 4	Framework web para crear el servidor y los endpoints HTTP.
Docker Hub	Registro donde se publicó la imagen (usuario mpaula14).
Visual Studio Code y Terminal	Edición del código y ejecución de comandos Docker.
Google Chrome	Prueba del servicio expuesto en http://localhost:3000.

5. Desarrollo del laboratorio

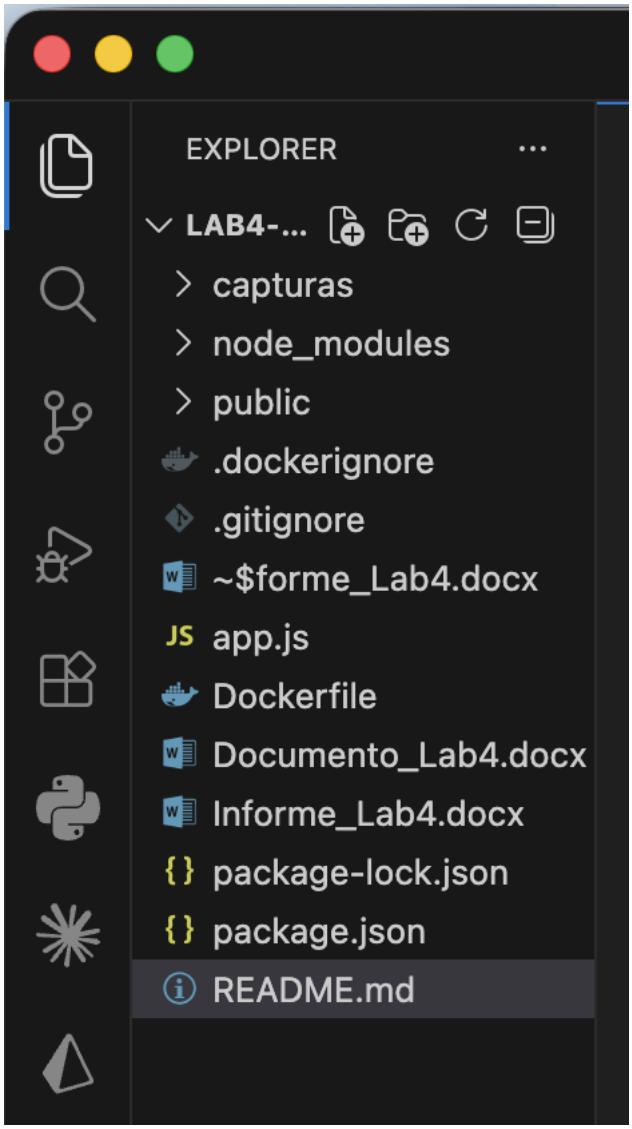
5.1 Creación de la aplicación

Se creó una aplicación web en Node.js con Express. El servidor escucha en el puerto 3000 y ofrece una interfaz web interactiva (una calculadora) junto con los siguientes endpoints:

Endpoint	Descripción	Ejemplo de respuesta
GET /	Interfaz web interactiva (calculadora).	Página HTML

Endpoint	Descripción	Ejemplo de respuesta
GET /saludo	Mensaje de bienvenida.	¡Hola desde Docker!
GET /calcular?op=sumar&a=5&b=3	Suma, resta, multiplicación o división de a y b, con validación de datos y de división entre cero.	{"resultado": 8}
GET /health	Estado del servicio e ID del contenedor.	{"estado": "ok"}

La estructura del proyecto es la siguiente:



Fragmento principal del endpoint de cálculo (app.js):

```
app.get('/calcular', (req, res) => {
  const { op } = req.query;
  const a = Number(req.query.a);
  const b = Number(req.query.b);
  const operaciones = {
    sumar: () => a + b,
    restar: () => a - b,
    multiplicar: () => a * b,
    dividir: () => (b === 0 ? null : a / b),
  };
  ...
  res.json({ operacion: op, a, b, resultado });
});
```

5.2 Creación del Dockerfile

Se escribió el siguiente Dockerfile para construir la imagen de la aplicación:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Instrucción	Función
FROM node:20-alpine	Usa como base la imagen oficial de Node.js 20 sobre Alpine Linux, que es muy liviana.

Instrucción	Función
WORKDIR /app	Define /app como directorio de trabajo dentro del contenedor.
COPY package*.json ./	Copia primero los archivos de dependencias para aprovechar la caché de capas.
RUN npm install --omit=dev	Instala solo las dependencias de producción (Express).
COPY . .	Copia el código fuente de la aplicación.
EXPOSE 3000	Documenta que la aplicación escucha en el puerto 3000.
CMD ["node", "app.js"]	Comando que se ejecuta al iniciar el contenedor.

5.3 Construcción de la imagen

Desde la carpeta del proyecto se construyó la imagen con el nombre miapp-docker y la etiqueta v1.

```
cd ~/Documents/lab4-docker
docker build -t miapp-docker:v1 .
```

Como se observa en la Figura 1, Docker ejecutó los 5 pasos del Dockerfile y terminó exitosamente (FINISHED). Las capas marcadas como CACHED se reutilizaron de una construcción anterior, lo que muestra el beneficio de la caché por capas.

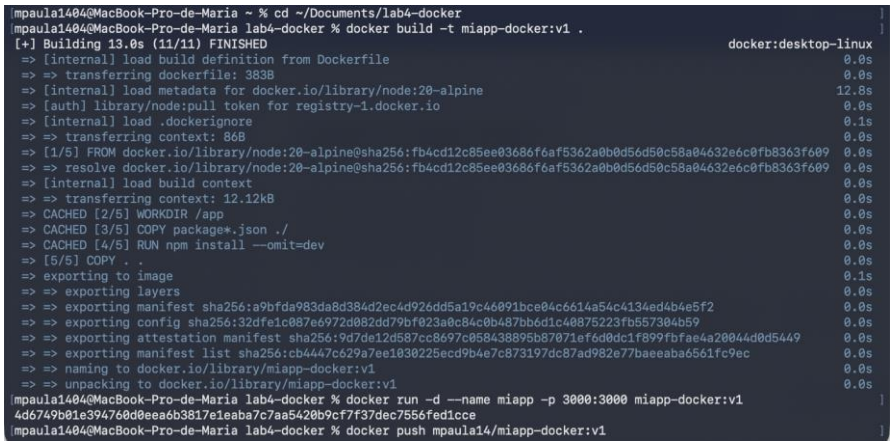


Figura 1. *Construcción de la imagen con docker build y ejecución del contenedor con docker run.*

5.4 Ejecución del contenedor

Se creó y ejecutó un contenedor a partir de la imagen:

```
docker run -d --name miapp -p 3000:3000 miapp-docker:v1
```

El parámetro **-d** ejecuta el contenedor en segundo plano, **--name** le asigna el nombre miapp y **-p 3000:3000** conecta el puerto 3000 del equipo con el puerto 3000 del contenedor. Docker devolvió el identificador del contenedor (4d6749b01e39…), y en Docker Desktop se verificó que estaba en ejecución con el puerto 3000:3000 publicado (Figura 2).

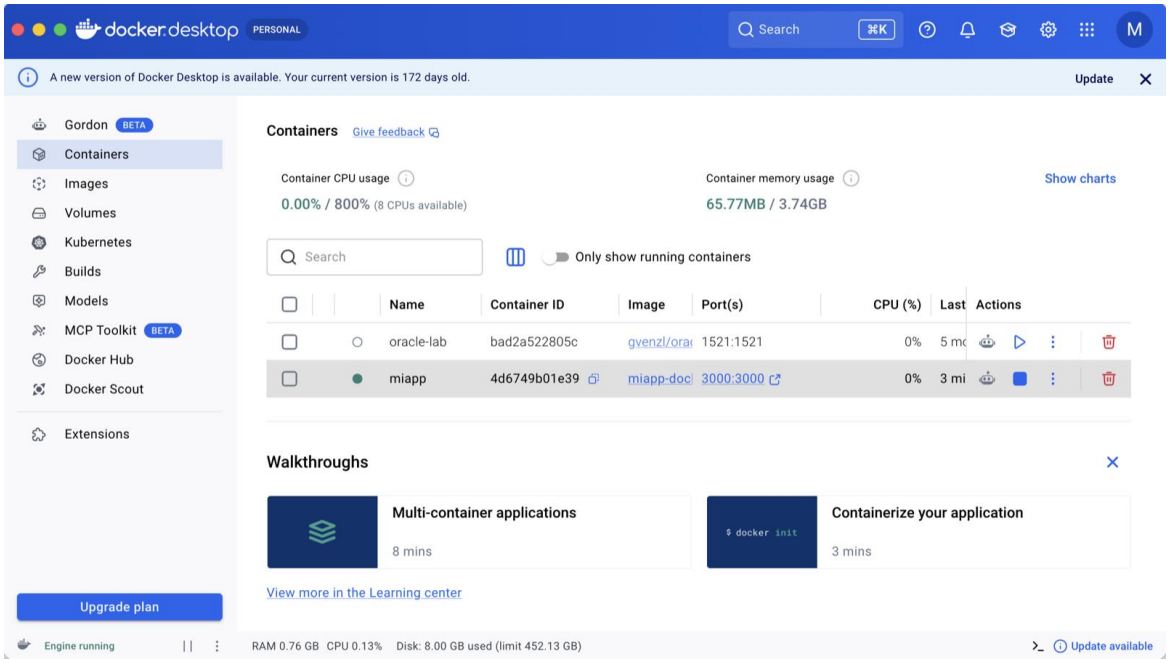


Figura 2. *Contenedor miapp en ejecución en Docker Desktop, con el puerto 3000:3000 expuesto.*

5.5 Prueba del servicio

Con el contenedor en ejecución se abrió http://localhost:3000 en el navegador. La interfaz realiza peticiones al endpoint /calcular y muestra el resultado; por ejemplo, 12 + 4 = 16. El indicador “Contenedor activo · 4d6749b01e39” coincide con el ID del contenedor que se ve en Docker Desktop, lo que confirma que la respuesta proviene del contenedor.

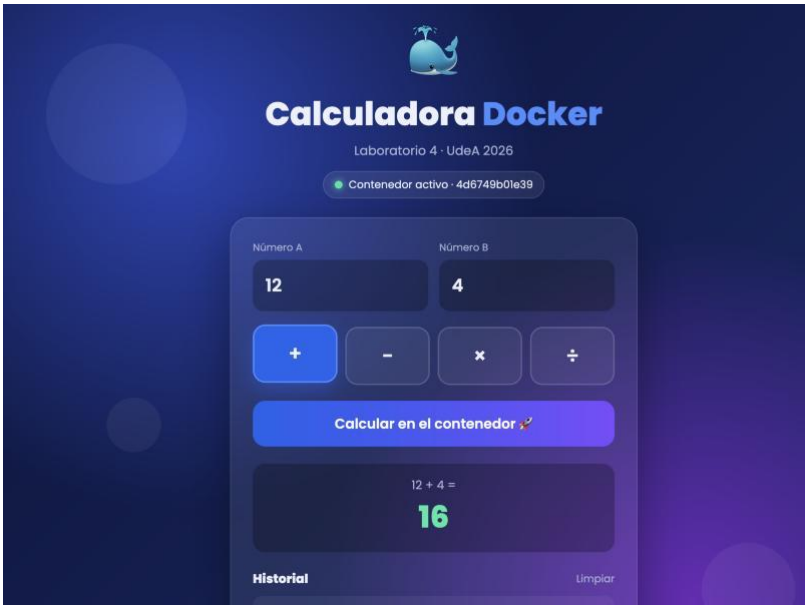


Figura 3. *Aplicación funcionando en el navegador desde el contenedor.*

5.6 Publicación en Docker Hub

Para publicar la imagen se etiquetó con el nombre de usuario de Docker Hub, se inició sesión y se subió al registro:

```
docker tag miapp-docker:v1 mpaula14/miapp-docker:v1
docker login
docker push mpaula14/miapp-docker:v1
```

La Figura 4 muestra que todas las capas de la imagen se subieron correctamente (Pushed) y que el registro devolvió el digest de la versión v1.

```
mpaula1404@MacBook-Pro-de-Maria lab4-docker % docker push mpaula14/miapp-docker:v1
The push refers to repository [docker.io/mpaula14/miapp-docker]
bda5d7ef971f: Pushed
c5bd562034f6: Pushed
83745518e77f: Pushed
7e4842b21d7c: Pushed
13e45b12880f: Pushed
cd322d0ddd02: Pushed
e8f424da7f51: Pushed
909f7e1923cc: Pushed
d17f077ada11: Pushed
v1: digest: sha256:5a05527ca8f7030b27d002c8c4daf339111d55a9c0047b29321acb08d023b544 size: 856
mpaula1404@MacBook-Pro-de-Maria lab4-docker %
```

Figura 4. *Publicación de la imagen con docker push.*

Finalmente, en Docker Hub se verificó que el repositorio mpaula14/miapp-docker quedó creado con visibilidad pública (Figura 5). Cualquier persona puede descargarlo y ejecutarlo con:

```
docker pull mpaula14/miapp-docker:v1
docker run -p 3000:3000 mpaula14/miapp-docker:v1
```

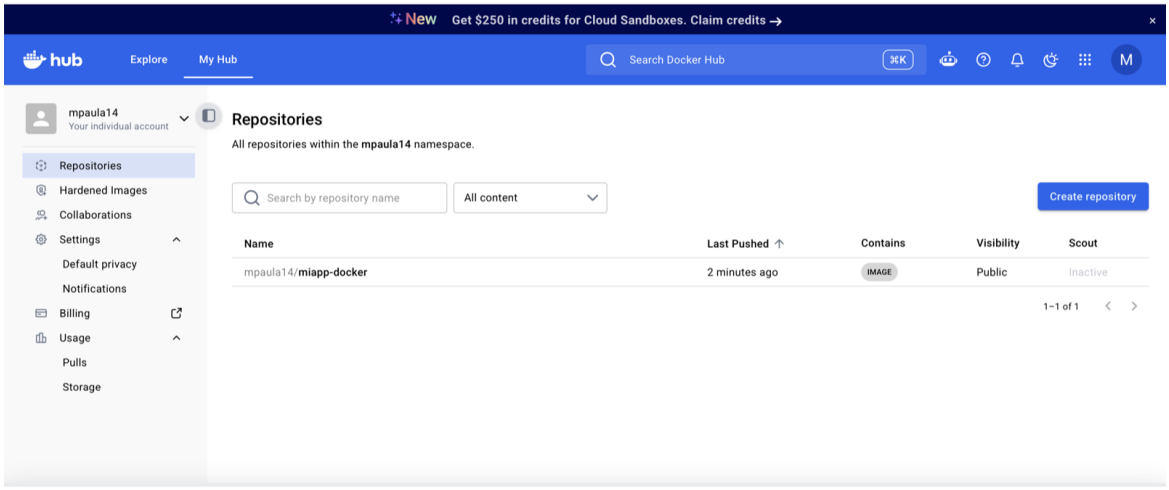


Figura 5. *Repositorio mpaula14/miapp-docker publicado en Docker Hub.*

6. Cómo Docker mejora la arquitectura de despliegue

- **Portabilidad:** la imagen incluye el entorno de ejecución y las dependencias, así que la aplicación funciona igual en un portátil, un servidor o la nube. Se elimina el problema de "en mi máquina sí funciona".
- **Aislamiento:** cada contenedor tiene sus propios procesos, red y sistema de archivos, de modo que aplicaciones con versiones distintas de librerías pueden convivir en el mismo equipo sin conflictos.
- **Reproducibilidad y versionamiento:** el Dockerfile funciona como documentación ejecutable, y las etiquetas (v1, v2…) permiten versionar la aplicación y volver a una versión anterior si es necesario.
- **Escalabilidad:** se pueden levantar varias réplicas idénticas en segundos, por ejemplo con Docker Compose, Docker Swarm o Kubernetes, detrás de un balanceador de carga.
- **Eficiencia:** al compartir el kernel del anfitrión, los contenedores usan menos memoria y disco que una máquina virtual y arrancan mucho más rápido.

- **Distribución e integración continua:** registros como Docker Hub facilitan compartir imágenes e integrarlas en pipelines de CI/CD que construyen, prueban y despliegan automáticamente.

7. Conclusiones

- Se logró empaquetar una aplicación web en Node.js dentro de una imagen Docker usando un Dockerfile sencillo y una imagen base liviana.
- Se comprendió la diferencia entre imagen (plantilla inmutable) y contenedor (instancia en ejecución), y cómo el mapeo de puertos permite acceder al servicio desde el equipo anfitrión.
- Ordenar las instrucciones del Dockerfile (dependencias antes que el código) permite reutilizar capas en caché y acelera las construcciones.
- La publicación en Docker Hub demostró que la aplicación puede distribuirse y ejecutarse en cualquier equipo con Docker, sin instalar Node.js ni dependencias adicionales.

8. Referencias

- Docker Inc. (2026). Docker Documentation. https://docs.docker.com
- Docker Inc. (2026). Dockerfile reference. https://docs.docker.com/reference/dockerfile/
- Docker Inc. (2026). Docker Hub. https://hub.docker.com
- OpenJS Foundation. (2026). Express — Node.js web application framework. https://expressjs.com